

CryptoChain Analyzer Dashboard

Hash Functions and Blockchain — Final Report
Cryptography | Universidad Alfonso X el Sabio
Prof. Jorge Calvo — Academic Year 2025–26

Amaia Martín Grande | GitHub: amg00005

1. Introduction

This report documents the CryptoChain Analyzer Dashboard, a real-time Python dashboard built with Streamlit that connects to the Bitcoin network via public APIs (Blockstream, blockchain.info) and displays live cryptographic metrics. The dashboard implements four required modules (M1–M4) and two optional modules (M6), covering Proof of Work mechanics, block header inspection, difficulty history, anomaly detection using machine learning, and a 51% attack cost estimator.

2. Required Modules (M1–M4)

2.1 M1 · Proof of Work Monitor

M1 displays live data about the current state of Bitcoin mining. The current difficulty is shown alongside its visual representation as a leading-zero threshold in the 256-bit SHA-256 space. The field **bits** in the block header encodes the target T in compact format (NBits): given $\text{bits} = 0xAABBCCDD$, the target is $T = 0xBBCCDD \times 256^{(AA-3)}$. A block hash is valid only if $\text{SHA256}(\text{SHA256}(\text{header})) < T$.

The distribution of inter-block times is plotted as a histogram alongside the theoretical exponential curve with $\lambda = 1/600$ s. This distribution is expected because Bitcoin mining is a memoryless Bernoulli process: each hash attempt is independent with a very low probability of success, making the waiting time exponentially distributed. The network hash rate is estimated using the formula: $\text{hashrate} = \text{difficulty} \times 2^{32} / 600$.

2.2 M2 · Block Header Analyzer

M2 parses the 80-byte Bitcoin block header field by field using Python's struct module (little-endian byte order). The six fields are: version (bytes 0–3), prev_block_hash (bytes 4–35), merkle_root (bytes 36–67), timestamp (bytes 68–71), bits (bytes 72–75), and nonce (bytes 76–79). Hash fields are stored in little-endian on the wire and reversed to big-endian for display.

The Proof of Work is verified locally using Python's hashlib: `hash = SHA256(SHA256(header_bytes))[:-1].hex()`. The result is compared against the decoded target to confirm $\text{hash} < \text{target}$. The module also visualises the nonce position within the 2^{32} search space and verifies the Merkle root by comparing the locally parsed value against the API response.

2.3 M3 · Difficulty History

M3 plots the evolution of Bitcoin mining difficulty over the last N adjustment periods (one period = 2016 blocks $\approx$ 2 weeks). Each adjustment event is marked on the chart. The ratio between actual block time and the 600-second target is shown per period: ratio > 1 means blocks were slower than expected (difficulty will decrease); ratio < 1 means faster blocks (difficulty will increase). The adjustment formula is: $\text{new_difficulty} = \text{old_difficulty} \times (\text{actual_time} / 1,209,600)$.

The current period status is also displayed, including an estimate of the next difficulty adjustment based on the average block time observed so far in the ongoing period.

2.4 M4 · AI Component — Anomaly Detector

Model chosen: Isolation Forest (scikit-learn, unsupervised). **Why:** inter-block times follow an exponential distribution (Poisson process), not a Gaussian one. Isolation Forest does not assume Gaussian data and is robust to the heavy tail of the exponential distribution. Since no labelled dataset of 'anomalous' blocks exists, an unsupervised method is required.

Training data: the last 150 Bitcoin blocks fetched in real time from the Blockstream API. Three features are used: (1) ibt — raw inter-block time in seconds; (2) log_ibt — natural log of the time, which compresses the exponential tail; (3) ibt_z — relative deviation from the 600 s target.

Metric	Value	Interpretation
Anomaly rate (IF)	~5%	Configurable via contamination parameter
Agreement with z-score baseline	~96%	High $\rightarrow$ model is not labelling arbitrarily
Precision proxy (TP/IF)	~25%	IF detects subtle anomalies the z-score misses
Score gap (anomaly vs normal)	>0.18	Good separability between classes

Table 1: M4 model evaluation metrics.

Limitations: without a ground-truth dataset of confirmed anomalous blocks, evaluation metrics are indicative rather than definitive. The z-score baseline serves as a proxy. Anomalies with very long inter-block times may indicate hash rate drops; anomalies with very short times may reflect mining pool propagation advantages.

3. Optional Module — M6 · Security Score

M6 estimates the cost of a 51% attack on Bitcoin using live network hash rate data. The attacker must control more than 50% of the total network hash rate. Using the Antminer S21 Pro as a reference miner (234 TH/s, 3,510 W, ~\$2,400), the required number of units is: $\text{units} = \text{ceil}(\text{target_TH/s} / 234)$. The total cost per hour includes electricity cost and hardware amortisation over 18 months.

Parameter	Value
Network hash rate (April 2026)	~970 EH/s
Miners needed for 51%	~2,115,467 units
Hardware cost	~\$5.08 B
Operational cost/hour	~\$763,000/h

Power required	~7.43 TW
----------------	----------

Table 2: 51% attack cost estimates (April 2026).

M6 also implements the Nakamoto (2008) confirmation depth model from §11. The probability of a successful attack given attacker hash fraction q and confirmation depth z is approximated as $P \approx (q/p)^z$ for $q < 0.5$. With 6 confirmations, even an attacker with 40% of the hash rate has a success probability below 0.1%, which justifies Bitcoin's standard of 6 confirmations for high-value transactions.

4. References

- Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. <https://bitcoin.org/bitcoin.pdf>
- Blockstream API documentation. <https://blockstream.info/api/>
- Mempool.space API documentation. <https://mempool.space/docs/api>
- Scikit-learn: Isolation Forest. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Bitcoin Wiki — Difficulty. <https://en.bitcoin.it/wiki/Difficulty>